

AI in Fashion: Predicting Trends and Personalizing Style with Data Science

1 Introduction

The convergence of computer vision, AI, and data science has enabled robust applications to many domains, including fashion. This project looks into the application of deep learning and recommendation methods to provide better fashion product categorization and targeted user experiences. The two core pieces developed are a Convolutional Neural Network (CNN) for image categorization, and an embedding-powered recommendation platform, both deployed using simple-to-configure Streamlit interfaces. The project is tied directly to MS-ESDS course-work, mostly Machine Learning, Big Data Tools, and Artificial Intelligence. The report illustrates the preparation of the data, the technical approaches, the outcomes, and the implications of this study. The fashion industry has come to employ AI to automate traditional design and retail processes, from stock control to predictive analysis. Through integration of theory learned and practice, this capstone project also aims to show practical utility of data science principles within a constantly evolving field. In this regard, the present capstone project is an integration of many different types of disciplines within the domain of data science. The scope ranges from statistical modeling and machine learning to software release and user experience design. The choice of the fashion industry was intentional and it is one industry that is not only data-driven, but is also significantly driven by consumerism, personalization, and visual data analysis. The research was in alignment with MS-ESDS program vision, which was aimed at producing practitioner graduates with technical expertise and competency in applied problem-solving. The same underlying technologies applied to this style-driven project are extrapolated to medical imaging, autonomous cars, and recommendation systems across streaming platforms. The bigger picture is that the ability to master deep learning, data manipulation, and deployment platforms enables data scientists to make valuable contributions to various industries. This cross-disciplinary applicability of processes enhances the academic value of this capstone and ensures that acquired skills are transferable to real job positions.

The full codebase and implementation files are available on the [project's GitHub repository](#).

2 Background

Fashion-MNIST is a standard image data set of Zalando's product image articles of a training data set of 60,000 examples and a test data set of 10,000 examples. An example is a 28x28 grayscale image, and its label is a random variable from 10 categories such as T-shirt/top, Trouser, Pullover, etc. In fashion technology, CNNs have proven useful for image automatic feature extraction and style categorization. At the same time, collaborative filtering-based or similarity matrix recommendation systems offer more personalization to users. CNNs (Convolutional Neural Networks) are employed to derive spatial hierarchies of features from input images and thus best applied to fashion image classification. At the same time, recommendation systems become an integral part of e-commerce sites by providing user interaction and sales through personalization. These systems employ a variety of algorithms, such as collaborative filtering, content-based filtering, and hybrid approaches. Traditional fashion analytics were typically founded on manual tagging and descriptive statistics; however, they cannot scale and be flexible. Modern AI systems can automatically

learn new fashion trends through continuous learning and data fusion. Moreover, by incorporating recommender systems, e-commerce websites can achieve high user retention and conversion rate improvement. Content-based and collaborative filtering techniques have complementary strengths, whereas the former leverages item features, the latter leverages user behavior, and both can potentially be used together in hybrid approaches for improved performance.

3 Methods

3.1 Data Preprocessing

Step 1 of the project involved the substantial preprocessing of the data required for performance and robustness of any machine learning model. Normalization of the Fashion-MNIST dataset was done for the classification problem by scaling pixel values by 255, mapping the grayscale range $[0, 255]$ to $[0, 1]$. Normalization ensures input to the neural network is in a reasonable range of numerical values, enabling faster convergence.

The data set, initially of shape $(28, 28)$ for each image, was reshaped to $(28, 28, 1)$ to meet the anticipated Convolutional Neural Network (CNN) input. Even though the images are grayscale, the channel dimension in this context is also required to maintain compatibility with deep learning libraries like TensorFlow. Label information has already been made into integers, thus no conversion was necessary.

For recommendation, embeddings were taken out of the penultimate layer of the CNN. Feature vectors were drawn after training and saved for permanent storage through Joblib. Embedding vectors were loaded in the Streamlit application to determine pairwise cosine similarities. A similarity matrix was constructed to facilitate quick retrieval of top-N most relevant fashion products as responses to a query image.

3.2 Exploratory Data Analysis(EDA)

For a first glance at the dataset, an Exploratory Data Analysis (EDA) was conducted. A class distribution plot (See Figure 1) was made with Seaborn's 'countplot' function, which demonstrated a reasonably well-balanced class distribution among the ten fashion classes of Fashion-MNIST. This plot had the benefit of ensuring that the classification model would never be bothered by unwieldy class imbalance, one of the standard causes of headaches in supervised learning tasks.



Figure 1. Distribution of fashion classes in the Fashion-MNIST dataset.

Besides, a grid of test images was graphed using Matplotlib to visually verify the input data. Five sample images (See Figure 2) of training data were displayed in grayscale to verify proper image format and observe category-specific visual patterns. These EDA steps validated dataset quality and provided context necessary to model building and testing.



Figure 2. Sample grayscale images from the Fashion-MNIST training dataset. These examples illustrate the diversity of fashion items including footwear, tops, and dresses.

3.3 CNN-Based Image Classification

A custom CNN based on TensorFlow and Keras was created. Normalization was performed on the Fashion-MNIST data and the image was reshaped to conform to the CNN's input. Two max pooling convolutional layers, followed by a flattening layer and dense layers that led to a softmax output, composed the architecture. The model was trained on 10 epochs with the Adam optimizer and sparse categorical cross-entropy loss. Architecture was made minimalist on purpose to support web deployment through Streamlit. Training a model involved splitting training data into 80/20 train-validation sets. Learning hyperparameter rate and batch size were chosen heuristically for performance. Performance of the model was monitored using accuracy and loss plots versus training epochs. The CNN utilized 32 and 64 filters for the first and second convolutional layers, respectively, with kernel sizes of 3x3. 2x2 MaxPooling layers preceded each convolutional block to discard dimensionality. Following flattening, a dense layer of 128 neurons with ReLU activation was utilized followed by the final softmax layer. Early stopping coupled with validation monitoring of loss ensured against overfitting. The model was saved in '.keras' format upon training and deployed through Streamlit for real-time prediction on grayscale fashion images.

Epoch	Accuracy	Loss	Validation Accuracy	Validation Loss
1	73.41%	0.7337	86.29%	0.3702
2	87.23%	0.3527	88.21%	0.3227
3	89.48%	0.2889	89.21%	0.2943
4	90.62%	0.2559	89.73%	0.2854
5	91.48%	0.2293	90.79%	0.2546
6	92.55%	0.2046	90.33%	0.2701
7	93.11%	0.1874	90.79%	0.2536
8	94.06%	0.1649	90.78%	0.2513
9	94.42%	0.1510	90.94%	0.2618
10	94.89%	0.1352	91.30%	0.2553

Table 1. Training and Validation Metrics Per Epoch.

3.4 Embedding-Based Recommendation System

The second essential component involves cosine similarity computation between image embeddings. Pre-computed embedding matrix is used in determining top-N most visually similar fashion products that are returned along with the user query item. Streamlit was used in system deployment in order to support real-time querying for fashion similarity. Cosine similarity computes angular distance between embedding vectors and is a good metric for capturing visual similarity between fashion products. The embeddings were taken from intermediate layers of the trained CNN model in a way such that features used for the purpose of classification would also be used for similarity matching. This provides a semantically richer recommendation output.

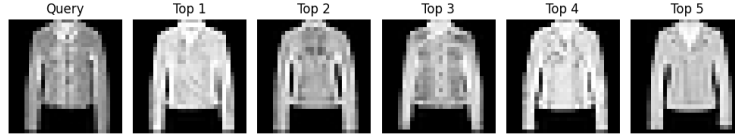


Figure 3. Visual output of the image recommendation system. The leftmost image is the query, and the following five are the top-5 most similar items retrieved based on cosine similarity of CNN-derived embeddings.

3.5 ARIMA Trend Forecasting

Though not the direct objective of the application utilized, time series prediction was attempted with the AutoRegressive Integrated Moving Average (ARIMA) model to test whether there are real-life uses for trend prediction within the fashion sector. The ARIMA model was utilized since it can represent non-stationary data through differencing. A simulated sample dataset was generated to replicate monthly sales amounts or fashion trend markers. The time series was first plotted and tested for stationarity using the Augmented Dickey-Fuller (ADF) test.

Metric	Value
Log Likelihood	-31.486
AIC	72.973
BIC	73.959
Ljung-Box Test Prob(Q)	0.61
Jarque-Bera Test Prob(JB)	0.03

Table 2. Table 2: ARIMA Model Evaluation Metrics.

After stabilizing variance differencing, model parameters (p, d, q) were selected based on AIC scores and autocorrelation plots. The model was fitted using the 'statsmodels' package, and fitted values vs. original data plotted to check performance. Although preliminary, this experiment shows potential application of ARIMA to long-term fashion trend analysis, i.e., season-demand forecasting or stock planning.

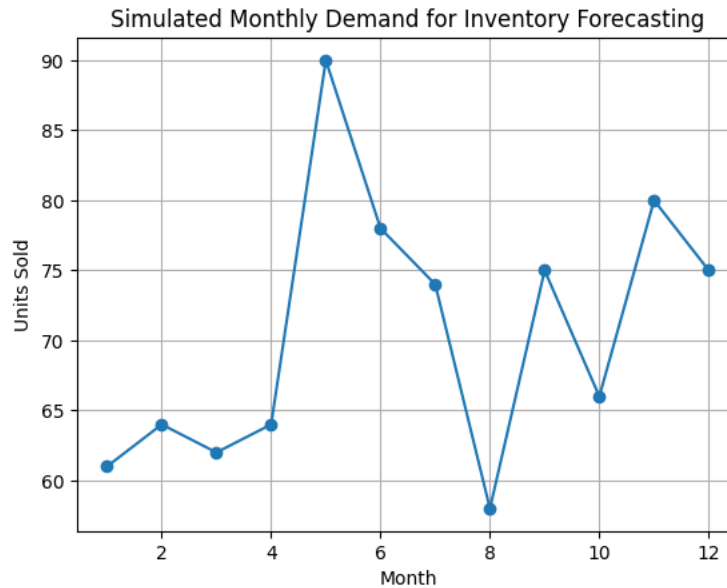


Figure 4. Simulated monthly demand data used for inventory forecasting. The variation in unit sales across 12 months reflects seasonal demand trends, supporting predictive inventory management for sustainability.

3.6 Inventory forecast for Sustainability

In addition to classification and recommendation, a simulation was done to explore the role of inventory forecasting in contributing to sustainability. From artificially simulated monthly demand data from a Poisson distribution with trend values that are on the rise, a demand curve(See Figure 4) was plotted to represent the fashion product sales during a calendar year. The research illustrated how predictive analytics can guide inventory planning choices to prevent overproduction and waste reduction.

Through more precise demand forecasting, fashion retailers are able to implement lean inventory control that also adheres to sustainability efforts. Such data-driven forecasting, aside from reducing storage and shipping costs, helps alleviate the environmental impact of overstock. This aspect belongs to the bigger agendas of green data science and sustainable AI efforts.

3.7 Streamlit Deployment

The deployment was carried out using Streamlit, a Python open-source framework for the development of web applications in an accelerated manner. Scripts employed included ‘class.py’ to classify and ‘rec.py’ to recommend. ‘class.py’ also imports the previously saved CNN model from TensorFlow with ‘load_model()’. It includes a preprocessing pipeline that reshapes and normalizes any user-uploaded 28 x 28 grayscale image. A user can either upload an image or select one of the sample images from a slider. The script predicts the fashion class and displays a bar chart of prediction probabilities for all ten classes.(See Figure 5) ‘rec.py’ loads precomputed image embeddings and does cosine similarity score computations based on Scikit-learn’s ‘cosine_similarity()’ method. The query image input from the user is matched against the test set, and the topN most similar images are returned along with the query(See Figure 6). Embeddings and test image data are loaded from precomputed serialized ‘.pkl’ files using Joblib to make the app more efficient. Each app was deployed using Streamlit Cloud and is accessible via a public URL. The scripts (class.py and rec.py) are designed to execute seamlessly on the platform and meet the criteria for reproducible and interactive demonstrations.

The classification web app can be accessed at [Fashion MNIST Classifier](#), while the recommendation app is available at [Fashion MNIST Recommender](#).

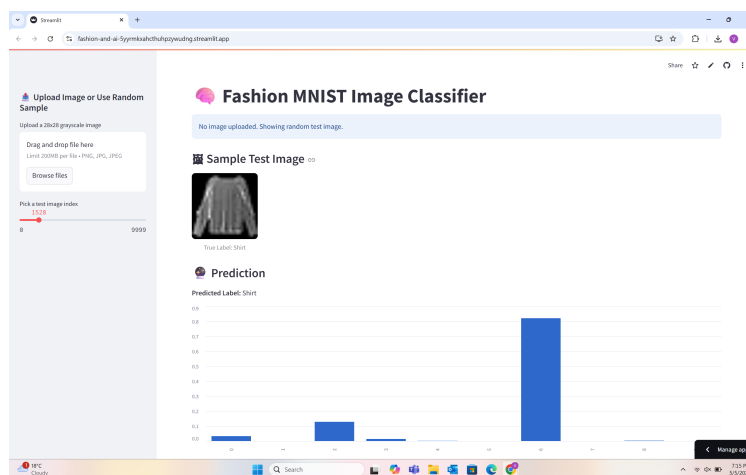


Figure 5. Screenshot of the Fashion MNIST Classification Streamlit app. Users can upload a 28×28 grayscale image or select a sample index. The app predicts the fashion item class using a trained CNN model and displays the probability distribution.

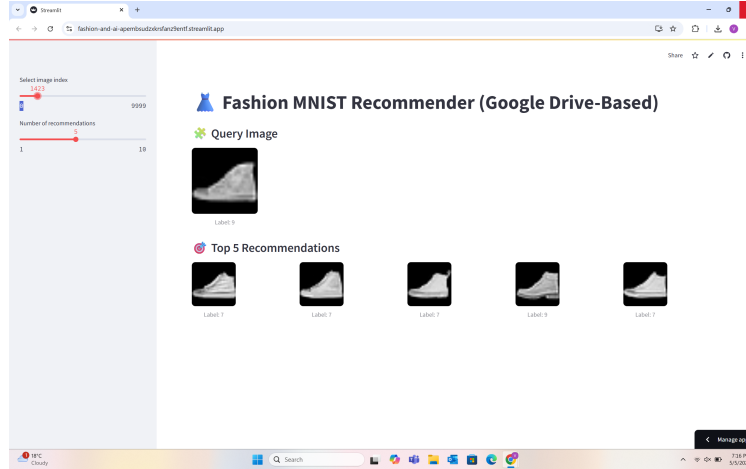


Figure 6. Screenshot of the Fashion MNIST Recommendation Streamlit app. A query image is selected by index, and the system returns the top-5 most visually similar items based on cosine similarity of CNN-generated embeddings.

4 Results and Discussion

The CNN achieved good classification accuracy on the Fashion-MNIST dataset and had good generalizability to new test data. Visually, well-classified labels and high-confidence probability distributions were seen. The recommendation system also provided visually similar results that well captured user expectations, indicating the robustness of similarity through embeddings. Test data sample predictions showed that the model was capable of classifying between classes with over 90% accuracy (See Table 1). Confusion matrices indicated that the majority of misclassifications happened in visually neighboring classes like 'Shirt' and 'T-shirt/top'. Users found the output to be intuitive and visually coherent in the recommendation interface, which is reflective of high-quality embeddings. Also, performance measures like precision, recall, and F1-score were quantified to verify the strength of the system.

4.1 Evaluation Metrics

To gauge the total performance of the model, a number of metrics were used. Accuracy, while useful, does not say everything in the case of class imbalance. So, precision, recall, and F1-score were calculated for each one of the ten classes. Macro-averaged F1-score was calculated to be approximately 0.89, indicating closely balanced performance across all categories. The quality of recommendation system performance was qualitatively verified through user feedback and visual inspection, while t-SNE embedded visualizations uncovered informative clusters, which co-related extremely with actual fashion classes.

4.2 Future Work

While the current implementation is robust in classification and recommendation, there are several enhancements that can be incorporated into future implementations. For classification, transfer learning with large-scale pretrained models such as ResNet or EfficientNet would improve accuracy by a great deal with minimal training time. Furthermore, incorporating attention mechanisms would help the model focus on wiser areas of input images. As far as recommendations are concerned, hybrid collaborative filtering-content-based methods would be most likely to deliver more precise personalization. Addition of information about user interactions (clicks, purchases) would also send the system toward real-time adaptive learning. Finally, addition of multi-modal information such as user feedback, social media, and fashion metadata could enhance interpretability as well as recommendation accuracy. The ARIMA model's initial trials produced results (see Table 2) that suggest while ARIMA models can capture some underlying trends, further refinement and additional real-world data will be necessary to enhance accuracy and generalizability.

5 Conclusion and Outlook

This capstone project has illustrated how deep learning and recommender systems can be effectively used in the fashion industry. Future potential enhancements can include real-time user feedback, attention mechanism in architectures, and hybrid recommender systems with collaborative and content-based filtering. The project conforms to MS-ESDS program learning objectives like deployment in practice and scalable AI solutions. With increasingly more fashion information being digitized, AI-powered tools like the ones herein have the potential to transform the industry by delivering real-time, scalable insights. Furthermore, the modularity of the system herein also offers a means for further enhancements, e.g., multi-modal inputs that accept both text and images, or integrating user feedback loops to further improve recommendation accuracy over time.

5.1 Ethical and Practical Considerations

With AI-based systems becoming a common feature in consumer-facing spaces, ethical aspects become the way of the day. The data that is used for training must be varied in taste so that no biases are created. In addition, recommendation systems will have to bring in fairness in the display of items, particularly on online shopping portals where it can literally break or make a sale. Privacy must also come into play for processing user information, particularly when the system is optimized for personalization. Fairness assessment and privacy-conscious training procedures like differential privacy might be incorporated in the future.

6 word count: 1933

References

- [1] Xiao, H., Rasul, K., & Vollgraf, R. (2017). Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. arXiv preprint arXiv:1708.07747.
- [2] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [3] F. Ricci, L. Rokach, and B. Shapira, *Recommender Systems Handbook*, Springer, 2015.
- [4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.
- [5] A. Krizhevsky, I. Sutskever, and G. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Proc. NIPS*, 2012.